# DEUKSIL — README CONFIGURATION

**Le seul document à suivre pour configurer DEUKSIL sur Supabase, de bout en bout.** Toutes les commandes, tous les réglages, tous les contrôles sont ici, dans l'ordre. Les autres documents du dossier ( `README_SUPABASE.md` , `docs/` ) ne servent qu'à approfondir ; ils ne sont pas nécessaires pour réussir la configuration.

Version du 22 octobre 2026.

---

## Sommaire


| Partie | Contenu | Durée indicative |
|---|---|---|
| 0. Avant de commencer | règles, vocabulaire, ce qui est déjà fait | 5 min |
| 1. Installer les outils | Supabase CLI, Node.js, client PostgreSQL, Playwright | 20 min |
| 2. Créer le projet Supabase de TEST | création, récupération des clés | 10 min |
| 3. Le fichier .env | ranger ses valeurs une fois pour toutes | 5 min |
| 4. Régler l'API et l'authentification | réglages par clics dans le tableau de bord | 20 min |
| 5. Créer la base de données | 8 migrations en une commande | 5 min |
| 6. Activer les rappels quotidiens | tâche planifiée | 5 min |
| 7. Déployer la fonction serveur | `gerer-acces` , `assistant-ia` | 5 min |
| 8. Brancher et publier le site | `js/00-config.js` , hébergement | 15 min |
| 9. Tester | tests automatiques + recette manuelle | 1 h |
| 10. Sauvegarde et restauration d'essai | une fois, avant toute vraie donnée | 30 min |
| 11. Passer en PRODUCTION | nouveau projet, administrateur, sécurité | 1 h |
| 12. Checklist finale | à cocher avant l'ouverture | 10 min |
| 13. Exploitation courante | sauvegardes, surveillance, évolutions | — |
| 14. En cas de problème | erreurs fréquentes et solutions | — |
| Annexe A | où ranger chaque secret | — |
| Annexe B | contenu du dossier | — |


---

## 0. Avant de commencer

### Trois règles

1. **On commence TOUJOURS par un projet Supabase de TEST.** La production ne vient qu'à la partie 11, quand tout a été validé en test.

2. **Aucun secret** ne va dans le code, dans Git, dans un e-mail, un message ou un ticket. Les secrets sont : la clé `service_role`, le mot de passe de la base, le mot de passe SMTP, le secret Google. Ils vont uniquement dans le tableau de bord Supabase et dans votre fichier `.env` local (voir partie 3).
3. **Un contrôle par étape.** Chaque étape se termine par un encadré ✅ **Contrôle**. Si le contrôle n'est pas bon : s'arrêter, consulter la partie 14, ne pas continuer.

### Vocabulaire

| Mot | Signification |
|---|---|
| **Projet Supabase** | un serveur complet chez Supabase : base de données, comptes, fichiers. On en crée deux : `deuksil-test` et `deuksil-prod`. |
| **ref** | l'identifiant du projet, 20 lettres (ex. `abcdefghijklmnopqrst`), visible dans l'adresse `https://<ref>.supabase.co`. |
| **Clé anon** (ou *publishable*) | clé **publique** : le site l'utilise dans le navigateur. Sans danger : la base vérifie elle-même les droits. |
| **Clé service_role** (ou *secret*) | clé **SECRÈTE** qui contourne toutes les protections. Uniquement côté serveur et dans `.env`. |
| **Migration** | un fichier SQL qui crée ou modifie la base. Il y en a 8, appliquées dans l'ordre. |
| **RLS** | *Row Level Security* : les règles qui font que chaque utilisateur ne voit que ses données. |
| **Terminal** | la fenêtre où l'on tape les commandes (Linux, macOS ; sous Windows : **WSL Ubuntu**). |

### Ce qui est déjà fait (vous n'avez rien à écrire)

- les **8 scripts SQL** de la base ( `supabase/migrations/` ) ;
- la **fonction serveur** `gerer-acces` ( `supabase/functions/gerer-acces/` ) ;
- les **modèles d'e-mails** ( `supabase/templates/` ) ;
- les **scripts de sauvegarde et restauration** ( `supabase/sauvegarde/` ) ;
- les **tests** automatiques ( `supabase/tests-local/` , `tests/` ) — livrés à part dans `DEUKSIL_OUTILS_DE_TEST.zip` (partie 9) ;
- le **site** complet.

Tout cela a été testé sur une copie locale de Supabase. Votre travail : **installer** tout cela dans **vos** projets Supabase et **vérifier** que tout fonctionne.

### Ce que vous allez remplir (et rien d'autre)

- le fichier **`.env`** (vos valeurs, partie 3) ;
- le fichier **`js/00-config.js`** (branchement du site, partie 8) ;
- des **réglages par clics** dans le tableau de bord Supabase (partie 4).

---

## 1. Installer les outils

Toutes les commandes se tapent dans un terminal, **depuis le dossier `DEUKSIL`** (celui qui contient ce fichier).

```
cd chemin/vers/DEUKSIL
```

> **Windows** : installer WSL ( `wsl --install` dans PowerShell administrateur, puis redémarrer) et faire tout le reste dans le terminal **Ubuntu**.

### 1.1 Node.js (version 18 ou plus)

```
node --version            # si absent ou < v18 :
curl -fsSL https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
source ~/.bashrc
nvm install 22
node --version            # doit afficher v22.x
```

### 1.2 Supabase CLI

```
npm install -g supabase        # ou, sur macOS : brew install supabase/tap/supabase
supabase --version
supabase login                 # ouvre le navigateur : se connecter au compte Supabase de
l'équipe
```

### 1.3 Client PostgreSQL (psql, pg_dump)

Sa version doit être **égale ou supérieure** à celle du serveur Supabase (15 ou 17 selon le projet ; on la vérifie à la partie 2).

```
# Ubuntu / WSL : dépôt officiel PostgreSQL, puis le client 17 (fonctionne aussi avec un
serveur 15)
sudo apt install -y curl ca-certificates
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc --fail
https://www.postgresql.org/media/keys/ACCC4CF8.asc
echo "deb [signed-by=/usr/share/postgresql-common/pgdg/apt.postgresql.org.asc]
https://apt.postgresql.org/pub/repos/apt $(. /etc/os-release && echo $VERSION_CODENAME)-pgdg
main" | sudo tee /etc/apt/sources.list.d/pgdg.list
sudo apt update && sudo apt install -y postgresql-client-17

# macOS
brew install libpq && brew link --force libpq

psql --version && pg_dump --version
```

### 1.4 Dépendances des scripts et des tests

```
npm install --no-save @supabase/supabase-js@2.45.4 pg@8 playwright@1
npx playwright install --with-deps chromium
(cd supabase/sauvegarde && npm install)
```

> ✅ **Contrôle** : `node --version`, `supabase --version`, `psql --version`, `pg_dump --version` répondent ; `npx playwright --version` répond.

---

## 2. Créer le projet Supabase de TEST

1. Aller sur **supabase.com** → se connecter → **New project**.
2. Remplir : * **Name** : `deuksil-test` * **Database Password** : cliquer *Generate a password*, le **copier dans le gestionnaire de mots de passe de l'équipe** (vous en aurez besoin pour le `.env`). * **Region** : la plus proche des utilisateurs parmi celles proposées (noter laquelle).
3. **Create new project**, attendre 1 à 2 minutes.
4. Récupérer les valeurs (les libellés peuvent légèrement varier selon la version du tableau de bord) :

| Valeur | Où la trouver |
|---|---|
| **ref** et **Project URL** | Project Settings → **API** (ou bouton **Connect**) : `https://<ref>.supabase.co` |
| **Clé anon / publishable** | Project Settings → **API** → *Project API keys* → `anon` `public` (ou *Publishable key*) |
| **Clé service_role / secret** | même page → `service_role` `secret` (cliquer *Reveal*) — **SECRÈTE** |
| **Chaîne de connexion** | bouton **Connect** (en haut) → *Connection string* → **URI** ; remplacer `[YOUR-PASSWORD]` par le mot de passe de l'étape 2 |

> **Connexion directe ou « pooler » ?** L'adresse directe `db.<ref>.supabase.co:5432` peut n'être joignable qu'en IPv6. Si `psql` échoue plus loin avec une erreur réseau, reprendre dans **Connect** la chaîne **Session pooler** (de la forme `postgresql://postgres.<ref>:MOTDEPASSE@aws-0-<région>.pooler.supabase.com:5432/postgres`).

5. Relier le dossier au projet :

```
supabase link --project-ref <ref>      # demande le mot de passe de la base
```

> ✅ **Contrôle** : `supabase projects list` affiche `deuksil-test` avec un lien (●) ; dans le tableau de bord → SQL Editor, `select version();` affiche la version de PostgreSQL (notez-la : votre `psql` doit être ≥).

---

## 3. Le fichier .env

On range **une fois** les valeurs du projet, pour que toutes les commandes de ce document fonctionnent **sans rien recopier**.

```
cp .env.example .env.test
```

Ouvrir `.env.test` dans un éditeur de texte et le remplir (exemple) :

```
SUPABASE_URL=https://abcdefghijklmnopqrst.supabase.co
SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....          # clé anon (publique)
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....  # clé service_role
(SECRÈTE)
DATABASE_URL=postgresql://postgres:MOT-DE-
PASSE@db.abcdefghijklmnopqrst.supabase.co:5432/postgres
DEUKSIL_ORIGINES=https://test.votre-domaine.sn       # adresse où le site de TEST sera publié
APP_URL=https://test.votre-domaine.sn/               # même adresse, avec / à la fin
MOT_DE_PASSE_TEST=Choisi-Par-Equipe-2026             # mot de passe des comptes du jeu de
test
DEUKSIL_TEST_AUTORISE=1                              # autorise les tests sur ce projet de
TEST
```

Les lignes SMTP et Google de `.env.example` peuvent rester vides : ces valeurs se saisissent dans le tableau de bord (partie 4).

**À chaque nouvelle séance de travail**, charger le fichier dans le terminal :

```
set -a; source .env.test; set +a
echo $SUPABASE_URL          # vérifie que c'est bien chargé
```

> `.env.test` et `.env.prod` sont ignorés par Git ( `.gitignore` ). Ne jamais les envoyer, les copier dans un ticket ou les mettre dans le site.
>
> ✅ **Contrôle** : `psql "$DATABASE_URL" -c "select 1"` affiche `1` .

---

## 4. Régler l'API et l'authentification (tableau de bord)

Tout se fait par clics, dans le projet `deuksil-test` . Cocher au fur et à mesure.

### 4.1 API

| ☐ | Où | Réglage |
|---|---|---|
| ☐ | Project Settings → **Data API** (ou API) → *Exposed schemas* | **uniquement** `public` (retirer tout autre schéma s'il y en a) |
| ☐ | même page → *Max rows* | **1000** |

### 4.2 Adresses du site

| ☐ | Où | Réglage |
|---|---|---|
| ☐ | Authentication → **URL Configuration** → *Site URL* | l'adresse du site de test, ex. `https://test.votre-domaine.sn` |

| ☐ | Où | Réglage |
|---|---|---|
| ☐ | même page → *Redirect URLs* → *Add URL* | `https://test.votre-domaine.sn/**` |

## 4.3 Inscription par e-mail et mots de passe

| ☐ | Où | Réglage |
|---|---|---|
| ☐ | Authentication → **Sign In / Providers** → **Email** | *Enable Email provider* : **activé** |
| ☐ | même écran | *Confirm email* : **activé** |
| ☐ | même écran | *Secure email change* : **activé** |
| ☐ | même écran (ou Authentication → *Policies / Passwords*) | *Minimum password length* : **8** ; *Password requirements* : **lettres et chiffres** |
| ☐ | même écran | *Email OTP Expiration* : **900** secondes |
| ☐ | Authentication → Sign In / Providers | *Allow anonymous sign-ins* : **désactivé** |

## 4.4 Envoi des e-mails (SMTP)

Le service d'envoi intégré de Supabase est limité à quelques e-mails par heure et réservé aux essais : il faut un **SMTP personnalisé** (Brevo, Mailjet, Amazon SES, Google Workspace, Zoho…).

| ☐ | Où | Réglage |
|---|---|---|
| ☐ | Authentication → **Emails** → **SMTP Settings** → *Enable custom SMTP* | activé |
| ☐ | même écran | *Sender email* : `noreply@votre-domaine.sn` ; *Sender name* : `DEUKSIL` |
| ☐ | même écran | *Host*, *Port* (587 en général), *Username*, *Password* : fournis par votre fournisseur SMTP (**le mot de passe est SECRET**) |

## 4.5 Modèles d'e-mails (code à 6 chiffres)

Les modèles sont dans `supabase/templates/`. Afficher leur contenu pour le copier :

```
cat supabase/templates/confirmation.html
cat supabase/templates/recovery.html
```

| ☐ | Où | Objet (*Subject*) | Contenu (*Body*) : coller |
|---|---|---|---|
| ☐ | Authentication → **Emails** → **Templates** → *Confirm signup* | `Confirmez votre adresse DEUKSIL` | tout `confirmation.html` |
| ☐ | même écran → *Reset password* | `Votre code DEUKSIL` | tout `recovery.html` |

## 4.6 Connexion Google (facultatif — à laisser désactivé pour le test)

Si vous l'activez plus tard : Google Cloud Console → API et services → Identifiants → *Créer un ID client OAuth* (Application Web) → *URI de redirection autorisés* : `https://<ref>.supabase.co/auth/v1/callback`. Puis Supabase → Authentication → Sign In / Providers → **Google** : coller l'ID client et le secret (**SECRET**). Puis `googleActif: true` dans `js/00-config.js` (partie 8).

> ✅ **Contrôle** : chaque ligne des tableaux 4.1 à 4.5 est cochée. (L'envoi réel des e-mails sera vérifié à la partie 9.)

---

## 5. Créer la base de données (migrations)

Une seule commande applique les 8 scripts dans l'ordre :

```
supabase db push
```

La commande liste les 8 migrations et demande confirmation ( `Y` ). Elle peut demander le mot de passe de la base.

| Migration | Ce qu'elle crée |
|---|---|
| `20261022000100_base.sql` | extensions et outils de base |
| `20261022000200_tables.sql` | les 32 tables |
| `20261022000300_comptes_et_droits.sql` | création du profil à l'inscription, protection des comptes, cohérence des données |
| `20261022000400_rls.sql` | règles d'accès (chacun ne voit que ses données) |
| `20261022000500_fonctions_api.sql` | fonctions appelées par le site |
| `20261022000600_stockage.sql` | dossiers de fichiers : `photos` , `avatars` (publics), `documents` (privé) |
| `20261022000700_regles_et_rappels.sql` | numérotation des pièces, règles métier, rappels quotidiens |
| `20261022000800_performances.sql` | optimisations et 58 index |

**Vérifier** : coller dans le tableau de bord → **SQL Editor** → *Run* :

```
select
  (select count(*) from pg_tables where schemaname = 'public')                    as
tables_32,
  (select count(*) from pg_tables where schemaname = 'public' and not rowsecurity)  as
sans_rls_0,
  (select count(*) from pg_policies where schemaname = 'public')                  as
politiques_166,
  (select string_agg(id || ':' || public, ' ' order by id) from storage.buckets)    as
compartiments;
```

✅ **Contrôle** : `tables_32` = **32**, `sans_rls_0` = **0**, `politiques_166` = **166**, `compartiments` = `avatars:true documents:false photos:true`.

Si une migration échoue : lire le message (partie 14). Ne jamais modifier une migration déjà appliquée ; corriger par une **nouvelle** migration (`supabase migration new correction`).

---

## 6. Activer les rappels quotidiens (pg_cron)

Chaque matin, la base envoie les rappels (loyers dus ou en retard, factures en retard, contrats et documents qui expirent, visites du lendemain), passe les contrats échus en « expiré » et reconduit les baux si le bailleur l'a demandé.

1. Tableau de bord → **Integrations** → **Cron** → *Enable* (ou **Database → Extensions** → rechercher `pg_cron` → activer).
2. Planifier la tâche (SQL Editor) — sans effet si elle existe déjà :

```
select cron.schedule('deuksil-rappels', '0 7 * * *', 'select app.generer_rappels()');
```

3. Essai immédiat, **deux fois de suite** :

```
select * from app.generer_rappels();
```

✅ **Contrôle** : `select jobname, schedule, active from cron.job;` → `deuksil-rappels | 0 7 * * * | true`. La deuxième exécution de `generer_rappels()` affiche **0** partout (pas de doublon). 07:00 UTC = 07:00 à Dakar.

---

## 7. Déployer la fonction serveur (gerer-acces)

Cette fonction crée les comptes qu'un propriétaire ou une agence ouvre pour quelqu'un (membre d'équipe, locataire, propriétaire mandant). Elle seule utilise la clé `service_role`, côté serveur.

```
set -a; source .env.test; set +a
supabase functions deploy gerer-acces
supabase secrets set DEUKSIL_ORIGINES="$DEUKSIL_ORIGINES"
supabase secrets list            # doit montrer DEUKSIL_ORIGINES
```

Ne **pas** définir `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY` : Supabase les fournit automatiquement à la fonction.

✅ **Contrôle** : `bash curl -s -o /dev/null -w "%{http_code}\n" -X POST "$SUPABASE_URL/functions/v1/gerer-acces"` doit afficher **401** (« connexion requise » : la fonction refuse les inconnus — c'est le bon comportement).

## 7.2 Assistant DEUKSIL (assistant-ia)

Relais vers Gemini pour l'assistant de recherche et la discussion ( `js/14-ai.js` ). La clé Gemini reste côté serveur. La fonction est ouverte aux visiteurs sans compte (déploiement `--no-verify-jwt` , comme dans `config.toml` ) : limiter `DEUKSIL_ORIGINES` au domaine du site et suivre le quota Gemini.

```
supabase functions deploy assistant-ia --no-verify-jwt
supabase secrets set GEMINI_API_KEY="$GEMINI_API_KEY"      # secret
supabase secrets set GEMINI_MODEL="${GEMINI_MODEL:-gemini-flash-latest}"
supabase secrets set GEMINI_PAYANT=oui     # SEULEMENT si la facturation Gemini est activée
(voir ci-dessous)
supabase secrets list            # doit montrer GEMINI_API_KEY et DEUKSIL_ORIGINES
```

> ✅ **Contrôle** : `bash curl -s -X POST "$SUPABASE_URL/functions/v1/assistant-ia" -H "apikey: $SUPABASE_ANON_KEY" -H "content-type: application/json" \ -d '{"mode":"chat","turns": [{"role":"system","content":"Réponds en une phrase."},{"role":"user","content":"Bonjour"}]}'` doit renvoyer `{"text":"…"}` . `{"message":"Assistant non configuré."}` = `GEMINI_API_KEY` manquante.

**Questions de gestion (factures, devis, proformas, contrats, loyers, impayés, aide à la décision)** : l'assistant reçoit alors un résumé des données que l'utilisateur connecté voit déjà dans son espace (droits des membres d'équipe respectés). La fonction n'accepte ce mode que si l'utilisateur est connecté (jeton vérifié par Supabase Auth) **et** si `GEMINI_PAYANT=oui` . Ne mettre `oui` qu'après avoir activé la **facturation** du projet Google AI Studio / Google Cloud de la clé : les conditions de l'offre gratuite interdisent d'envoyer des données personnelles ou confidentielles et autorisent Google à les utiliser pour améliorer ses produits ; l'offre payante ne les utilise pas. Sans `oui` , l'assistant répond aux questions générales et le signale.

**Outils de l'assistant dans les écrans** ( `js/99i-ia-outils.js` , mode `tache` de la fonction) :

| Outil | Écran | Offre payante exigée |
|---|---|---|
| Rédiger l'annonce (titre + description) | formulaire d'un bien | non |
| Estimer le prix (fourchette indicative, comparables DEUKSIL) | formulaire d'un bien | non |
| Suggestions de biens pour une demande client (anonymisée) | Agence → Clients | non |
| Analyser une annonce (prix anormal, texte copié, doublon, coordonnées) | Administration → Modération — rôle admin vérifié dans la base | non |
| Message de relance d'impayé (messagerie, WhatsApp, copie) | Gestion → Loyers | **oui** |
| Rapport du mois / relevé expliqué d'un propriétaire mandant | Gestion, Gestion → Rapports | **oui** |
| Résumer une conversation | Messagerie | **oui** |

| Outil | Écran | Offre payante exigée |
|---|---|---|
| Lire une facture ou un reçu (photo) et préremplir la dépense | Gestion → Dépenses | **oui** |

Chaque outil calcule d'abord ce qu'il peut sans IA (comparables, signaux, chiffres du mois, modèle de relance) : si l'assistant ne répond pas ou n'est pas activé, ce résultat reste affiché. Rien n'est enregistré sans un clic de l'utilisateur. Contrôle : un propriétaire de test voit les boutons « Rédiger avec l'assistant » et « Estimer le prix » dans le formulaire d'un bien.

---

# 8. Brancher et publier le site

## 8.1 `js/00-config.js` — le seul fichier du site à modifier

Ouvrir `js/00-config.js` et remplir :

```
const DEUKSIL_CONFIG = {
  supabaseUrl: 'https://abcdefghijklmnopqrst.supabase.co',   // = SUPABASE_URL
  supabaseAnonKey: 'eyJhbGciOi...',                          // = SUPABASE_ANON_KEY (JAMAIS
service_role)
  googleActif: false,                                        // true seulement si Google est
configuré (4.6)
  siteUrl: 'https://test.votre-domaine.sn',                  // = Site URL de la partie 4.2
  editeur: {                                                 // mentions légales (obligatoire
en production)
    raisonSociale: '', formeJuridique: '', adresse: '', ninea: '', rccm: '',
    directeurPublication: '', email: '', hebergeurSite: '',
    regionDonnees: ''                                        // région choisie à la partie 2
  }
};
```

Vérification qu'aucune clé secrète n'est dans le site :

```
grep -rn "service_role" js/
```

Résultat attendu : **une seule ligne**, le commentaire d'avertissement `js/00-config.js:6: NE JAMAIS mettre ici la clé service_role…`. Toute autre ligne = une clé secrète a été collée par erreur : la retirer.

## 8.2 Domaine Supabase personnalisé (seulement si vous en avez un)

Dans `index.html`, ligne `Content-Security-Policy`, ajouter votre domaine à `connect-src`, ex. `https://api.votre-domaine.sn wss://api.votre-domaine.sn`.

## 8.3 Publier

Copier chez l'hébergeur web (HTTPS obligatoire) **le contenu du dossier** sauf `supabase/`, `tests/`, `docs/`, les `.md`, `.env*` :

```
mkdir -p ../deuksil-site
cp -r index.html sw.js manifest.webmanifest css js img ../deuksil-site/
ls ../deuksil-site        # index.html  sw.js  manifest.webmanifest  css  js  img
```

Puis envoyer le contenu de `../deuksil-site` chez l'hébergeur (FTP, panneau d'hébergement, Netlify, Vercel, Cloudflare Pages, Nginx…), à l'adresse déclarée en 4.2.

> ✅ **Contrôle** : ouvrir l'adresse du site de test : la page d'accueil s'affiche **sans** « Configuration incomplète ». La console du navigateur (F12) ne montre pas d'erreur rouge.

---

## 9. Tester le projet de TEST

> ⚠️ **Jamais sur la production** : ces étapes créent des comptes et des données de test.
>
> 📦 **Les outils de test ne sont pas dans ce dossier** : le projet est livré **vide** (aucun compte, aucun bien, aucune donnée). Les tests automatiques et le jeu de données de test ( `supabase/tests-local/` , `tests/*.js` ) sont dans `DEUKSIL_OUTILS_DE_TEST.zip` , à décompresser à la racine du projet **uniquement sur le poste qui teste le projet Supabase de TEST**. Ne jamais publier ces fichiers sur l'hébergement.

### 9.1 Charger le jeu de données de test

29 comptes fictifs (adresses `@deuksil.test` ), biens, baux, loyers, factures… Tous les comptes ont le mot de passe `MOT_DE_PASSE_TEST` .

```
set -a; source .env.test; set +a
MOT_DE_PASSE="$MOT_DE_PASSE_TEST" node supabase/outils/importer-json.js \
  supabase/tests-local/jeu-de-test.json supabase/tests-local/comptes-test.json
```

> ✅ **Contrôle** : dernière ligne `29 comptes, … lignes importées, 2 rejetées …` (les 2 rejets sont deux notifications sans destinataire du jeu de test : normal).

### 9.2 Tests automatiques — dans cet ordre

```
node supabase/tests-local/securite-rls.test.js     # 1. sécurité de l'API
node tests/acces-comptes.js                        # 2. ouverture d'accès
node tests/sonde-pages.js                          # 3. toutes les pages de 9 profils
node tests/ecritures-auto.js                       # 4. aucune écriture automatique
```

| # | Résultat attendu |
|---|---|
| 1 | `65/65 tests réussis` (détail dans `supabase/tests-local/resultats/securite-rls.json` ) |

| # | Résultat attendu |
|---|---|
| 2 | `14/14 vérifications réussies` (la ligne « Requêtes refusées : 403 … rattacher_locataire » est **normale** : c'est un refus testé volontairement) |
| 3 | `### <profil> (… pages) OK` pour les 9 profils |
| 4 | seules des lignes de progression s'affichent ( `<profil> #/<page>` ) ; **aucune ligne commençant par `[`** (une telle ligne signalerait une écriture automatique) |

## 9.3 Recette manuelle des formulaires (15 min)

Les codes d'inscription et de mot de passe arrivent **par e-mail** : un test automatique ne peut pas les lire sur un projet hébergé. Une personne fait ce parcours avec **deux adresses e-mail auxquelles elle a accès** (A = client, B = agence ; astuce Gmail : `vous+client@gmail.com`, `vous+agence@gmail.com`), en navigation privée.

| # | Action | Résultat attendu | OK |
|---|---|---|---|
| 1 | Ouvrir le site | accueil complet | ☐ |
| 2 | « Publier un bien » | inscription, profil **Propriétaire** coché | ☐ |
| 3 | « Créer un compte agence » | profil **Agence** coché, champs agence visibles | ☐ |
| 4 | Inscription Client : e-mail `pas-une-adresse` | « Adresse e-mail invalide » | ☐ |
| 5 | Téléphone `12345` | refusé | ☐ |
| 6 | CNI `123` | « 13 chiffres » | ☐ |
| 7 | Mot de passe `abcdefgh` | refusé (chiffre manquant) | ☐ |
| 8 | Confirmation différente | « ne correspondent pas » | ☐ |
| 9 | Conditions non cochées | refusé | ☐ |
| 10 | Tout correct avec l'adresse **A** | page « Vérifiez votre adresse e-mail » | ☐ |
| 11 | Boîte A | e-mail « Confirmez votre adresse DEUKSIL », code à 6 chiffres, en moins de 2 min (voir les indésirables) | ☐ |
| 12 | Code `000000` | refusé | ☐ |
| 13 | Bon code | connecté, espace client | ☐ |
| 14 | Déconnexion → connexion, mauvais mot de passe | « incorrect » | ☐ |
| 15 | Bon mot de passe | connecté | ☐ |
| 16 | Déconnexion → « Mot de passe oublié ? » → adresse A | « Code envoyé » | ☐ |
| 17 | Boîte A | e-mail « Votre code DEUKSIL » | ☐ |
| 18 | « J'ai déjà un code » : code + nouveau mot de passe | « Mot de passe modifié », espace du compte | ☐ |
| 19 | Reconnexion avec le nouveau mot de passe | connecté ; l'ancien est refusé | ☐ |

| # | Action | Résultat attendu | OK |
|---|---|---|---|
| 20 | Inscription **Agence** avec l'adresse **B** (nom, NINEA…) + code | tableau de bord de l'agence | ☐ |
| 21 | Agence → « Ajouter un bien » → choisir **23** photos | **20** gardées (« 20/20 »), ajout désactivé | ☐ |
| 22 | Enregistrer le bien, le rouvrir | photos affichées | ☐ |
| 23 | Déconnecté : pied de page « Nous écrire », message `court` | « trop court » | ☐ |
| 24 | Message complet | « Message envoyé » | ☐ |

Vérifications en base (SQL Editor, remplacer les adresses) :

```
select role, cni, cgu_acceptees_le is not null as cgu from public.profiles where email =
'adresse-A';   -- client | … | true
select role, agency_name, ninea from public.profiles where email = 'adresse-B';
-- agency | nom | NINEA
select statut, subject from public.contact_messages order by created_at desc limit 1;
-- nouveau | …
```

### 9.4 Test d'injection de code (en DERNIER : il modifie les données)

```
node tests/xss-sonde.js
```

Attendu : `Aucune exécution ni élément injecté détecté.`

### 9.5 Remettre le projet de test à zéro (après 9.4)

```
supabase db reset --linked        # ⚠️ EFFACE TOUTE LA BASE du projet lié : vérifier que
c'est bien deuksil-test !
```

Puis refaire la partie 6 (planifier la tâche cron) si nécessaire.

> ✅ **Contrôle** : 9.1 à 9.4 conformes et les 24 lignes de la recette cochées.

---

## 10. Sauvegarde et restauration d'essai

À faire **une fois**, avant d'accueillir de vraies données : une sauvegarde qu'on n'a jamais restaurée ne garantit rien.

### 10.1 Sauvegarder le projet de test

```
set -a; source .env.test; set +a
cd supabase/sauvegarde
./sauvegarder.sh
```

Résultat : un dossier `supabase/sauvegarde/sauvegardes/AAAAMMJJ-HHMM/` contenant :

| Fichier | Contenu |
|---|---|
| `donnees.sql` | toutes les données, comptes compris |
| `structure.sql` | structure de la base (référence) |
| `fichiers/` + `manifeste.json` | photos, avatars, documents, avec empreintes |
| `comptages.txt` | nombre de lignes par table |
| `contexte.txt` | date, projet, version |
| `SHA256SUMS` | empreintes de contrôle |

Les messages `pg_dump: warning: there are circular foreign-key constraints…` sont **normaux** (gérés à la restauration).

> Ce dossier contient des **données personnelles** : le chiffrer puis le ranger hors du projet : `bash tar czf - sauvegardes/AAAAMMJJ-HHMM | gpg -c > deuksil-AAAAMMJJ-HHMM.tar.gz.gpg`

## 10.2 Restaurer dans un projet NEUF

1. Créer un projet vide `deuksil-restauration` (comme à la partie 2) et noter ses valeurs.
2. Y appliquer les migrations :

```
cd ../..                                  # retour au dossier DEUKSIL
supabase link --project-ref <ref-restauration>
supabase db push
```

3. Restaurer :

```
cd supabase/sauvegarde
DATABASE_URL_CIBLE="postgresql://postgres:MDP@db.<ref-
restauration>.supabase.co:5432/postgres" \
SUPABASE_URL="https://<ref-restauration>.supabase.co" \
SUPABASE_SERVICE_ROLE_KEY="<clé service_role de deuksil-restauration>" \
CONFIRMER=oui ./restaurer.sh sauvegardes/AAAAMMJJ-HHMM
```

Le script refuse de lui-même : sans `CONFIRMER=oui`, si la sauvegarde est altérée, si les migrations manquent, ou si la cible contient déjà des comptes (protection de la production).

> ✅ **Contrôle** : dernière ligne `VÉRIFICATION OK : mêmes comptages qu'à la sauvegarde.` Puis, sur le site branché temporairement sur ce projet (ou via l'API), un compte de test se connecte. Noter le **temps total** : c'est votre délai de remise en service.

4. Revenir au projet de test et supprimer le projet de restauration :

```
cd ../.. && supabase link --project-ref <ref-test>
```

Tableau de bord → `deuksil-restauration` → Project Settings → General → *Delete project*.

---

# 11. Passer en PRODUCTION

> Le projet de production part **vide**. Pour remettre à zéro un projet qui aurait reçu des données par erreur : sauvegarde d'abord, puis `psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -c "set deuksil.vider = 'oui'" -f supabase/outils/vider-projet.sql`, `CONFIRMER=oui node supabase/outils/vider-fichiers.js` (Storage), et contrôle avec `supabase/outils/verifier-projet-vide.sql` (toutes les lignes à 0).

## 11.1 Créer et configurer le projet

Refaire, sur un **nouveau** projet `deuksil-prod`, avec un fichier `.env.prod` :

| Partie | À refaire en production | Différence |
|---|---|---|
| 2 | créer `deuksil-prod`, `supabase link --project-ref <ref-prod>` | — |
| 3 | `cp .env.example .env.prod`, remplir | `DEUKSIL_TEST_AUTORISE=0`, `MOT_DE_PASSE_TEST` vide, adresses = site **réel** |
| 4 | tous les réglages | Site URL / Redirect URLs = adresse **réelle** |
| 5 | `supabase db push` | — |
| 6 | pg_cron + tâche | — |
| 7 | `supabase functions deploy gerer-acces` et `assistant-ia --no-verify-jwt` + `secrets set` | `DEUKSIL_ORIGINES` = adresse réelle ; `GEMINI_API_KEY` |
| 8 | `js/00-config.js` avec les valeurs **prod** + mentions légales **complètes** | publier sur l'adresse réelle |
| 9 | ❌ **NE PAS** importer le jeu de test ni lancer les tests automatiques | seulement un parcours manuel court (inscription, connexion) |

```
set -a; source .env.prod; set +a          # bien charger le fichier PROD
supabase link --project-ref <ref-prod>
supabase db push
supabase functions deploy gerer-acces
supabase functions deploy assistant-ia --no-verify-jwt
supabase secrets set DEUKSIL_ORIGINES="$DEUKSIL_ORIGINES" GEMINI_API_KEY="$GEMINI_API_KEY"
```

## 11.2 Premier administrateur

1. S'inscrire normalement sur le site de production avec l'adresse de l'administrateur, confirmer le code.
2. Lui donner le rôle administrateur (SQL Editor du projet **prod**) — seul ce moyen le permet, l'application et la fonction serveur le refusent :

```
update public.profiles set role = 'admin' where email = 'admin@votre-domaine.sn';
select email, role from public.profiles where role = 'admin';
```

3. Activer la double authentification de l'administrateur (application TOTP : Google Authenticator, etc.) depuis son espace, puis la rendre obligatoire pour les administrateurs :

```
update app.reglages set valeur = 'oui' where cle = 'admin_mfa_obligatoire';
```

> ⚠️ Faire l'étape 3 **seulement après** avoir activé la double authentification de l'administrateur, sinon il perd l'accès aux fonctions d'administration.

### 11.3 Sécurité de l'hébergement

Configurer chez l'hébergeur les en-têtes HTTP suivants (exemple Nginx ; chaque hébergeur a son écran « Headers ») :

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
```

Vérification : `curl -sI https://www.votre-domaine.sn | grep -iE "strict-transport|x-frame|nosniff|referrer"` → 4 lignes.

### 11.4 Première sauvegarde de production

```
set -a; source .env.prod; set +a
cd supabase/sauvegarde && ./sauvegarder.sh
```

---

## 12. Checklist finale avant ouverture

À cocher sur le projet **de production** :

| ☐ | Point | Comment vérifier |
|---|---|---|
| ☐ | Projet de test validé (parties 5 à 10 conformes) | — |
| ☐ | API : schéma exposé `public` seul ; Max rows 1000 | Project Settings → API |
| ☐ | Auth : confirmation e-mail, 8 caractères, SMTP personnalisé, 2 modèles, Site URL et Redirect URLs réels | partie 4 |
| ☐ | 8 migrations appliquées | `select version from supabase_migrations.schema_migrations order by 1;` → 8 lignes |
| ☐ | RLS partout | requête de la partie 5 → `sans_rls_0` = 0 |

| ☐ | Point | Comment vérifier |
|---|---|---|
| ☐ | Tâche `deuksil-rappels` active | `select * from cron.job;` |
| ☐ | Fonction `gerer-acces` déployée, `DEUKSIL_ORIGINES` défini | `curl … → 401` ; `supabase secrets list` |
| ☐ | Fonction `assistant-ia` déployée, `GEMINI_API_KEY` défini, `DEUKSIL_ORIGINES` = adresse réelle | contrôle de la partie 7.2 |
| ☐ | `js/00-config.js` : `siteUrl` = adresse **réelle** (pas `http://localhost:8080`) | lecture du fichier |
| ☐ | `js/00-config.js` : URL et clé **anon** de **prod**, `siteUrl`, mentions légales complètes | lecture du fichier |
| ☐ | Aucune clé `service_role` dans le site | `grep -rn service_role js/` → commentaire d'avertissement seulement |
| ☐ | En-têtes HTTP de sécurité | partie 11.3 |
| ☐ | Administrateur créé, MFA activée puis obligatoire | partie 11.2 |
| ☐ | Aucune donnée de test en production | `select count(*) from public.profiles where email like '%@deuksil.test';` → 0 |
| ☐ | Projet **vide** avant le premier administrateur | `psql "$DATABASE_URL" -f supabase/outils/verifier-projet-vide.sql` → 0 partout |
| ☐ | Sauvegarde de production faite, restauration testée (partie 10) | dossier chiffré rangé hors du projet |
| ☐ | Parcours manuel en production : inscription + code, connexion, publication d'un bien avec photo, message, déconnexion | — |

---

## 13. Exploitation courante

| Quand | Quoi | Commande / endroit |
|---|---|---|
| Chaque jour | la tâche des rappels a tourné | `select status, return_message, start_time from cron.job_run_details order by start_time desc limit 5;` |
| Chaque jour ou semaine | sauvegarde chiffrée hors du projet | `set -a; source .env.prod; set +a; cd supabase/sauvegarde && ./sauvegarder.sh` |
| Chaque trimestre | restauration d'essai | partie 10.2 |
| Chaque mois | index inutilisés, requêtes lentes | voir ci-dessous |
| À chaque évolution de la base | nouvelle migration → **test** → tests partie 9 → **prod** | voir ci-dessous |

**Faire évoluer la base** :

```
supabase migration new ajout_colonne_x          # crée
supabase/migrations/<date>_ajout_colonne_x.sql
# écrire le SQL dans ce fichier
supabase link --project-ref <ref-test> && supabase db push      # d'abord le TEST
node supabase/outils/generer-schema-front.js                    # met à jour js/02c-schema.js
(DATABASE_URL du test)
# tests de la partie 9, puis :
supabase link --project-ref <ref-prod> && supabase db push      # ensuite la PROD
```

**Surveiller les performances** (SQL Editor) :

```
-- requêtes les plus coûteuses
select calls, round(mean_exec_time::numeric, 1) as ms, left(query, 120) as requete
  from pg_stat_statements order by total_exec_time desc limit 20;
-- index jamais utilisés (après quelques semaines)
select relname, indexrelname, idx_scan from pg_stat_user_indexes where schemaname = 'public'
order by idx_scan limit 20;
```

---

## 14. En cas de problème

| Symptôme / message | Cause probable | Solution |
|---|---|---|
| Le site affiche « Configuration incomplète » | `js/00-config.js` non rempli | partie 8.1 |
| « Service momentanément indisponible » | mauvaise URL ou clé dans `00-config.js`, ou projet en pause | vérifier les valeurs ; tableau de bord → projet actif |
| `psql: … could not translate host name` / `Network is unreachable` | connexion directe IPv6 indisponible | utiliser la chaîne **Session pooler** (partie 2) |
| `password authentication failed` | mot de passe erroné dans `DATABASE_URL` | Project Settings → Database → *Reset database password*, mettre à jour `.env` |
| `supabase db push` : *Cannot find project ref* | dossier non lié | `supabase link --project-ref <ref>` |
| `PGRST202 Could not find the function … in the schema cache` | migration non appliquée | `supabase db push` |
| `42501 new row violates row-level security policy` | action non autorisée pour ce compte (c'est la sécurité qui fonctionne) | vérifier le rôle et les permissions du compte |
| `42501 Champs non modifiables : …` | tentative de modifier une colonne protégée d'un profil | normal : passer par la fonction prévue |
| `23P01 … baux_sans_chevauchement` | deux baux actifs sur le même logement aux mêmes dates | terminer l'ancien bail ou corriger les dates |

| Symptôme / message | Cause probable | Solution |
|---|---|---|
| `23505 duplicate key` | doublon (favori, créneau de visite, numéro) | normal ; recharger la page |
| `JWT expired` / `PGRST301` | session expirée | se reconnecter |
| Fonction : erreur **CORS** dans la console | `DEUKSIL_ORIGINES` ne contient pas l'adresse exacte du site | `supabase secrets set DEUKSIL_ORIGINES="https://…"` (sans `/` final) |
| Fonction : `curl` répond 404 | fonction non déployée | partie 7 |
| E-mails non reçus | SMTP non configuré ou refusé ; indésirables | partie 4.4 ; journaux : Authentication → Logs |
| Google : `redirect_uri_mismatch` | URI de redirection Google incorrecte | `https://<ref>.supabase.co/auth/v1/callback` |
| Rappels absents | pg_cron non activé / tâche absente | partie 6 |
| `securite-rls.test.js` : « Refus : … URL non locale » | `DEUKSIL_TEST_AUTORISE` absent | `.env.test` : `DEUKSIL_TEST_AUTORISE=1`, recharger |
| Tests navigateur : « Compte de test inconnu » | jeu de test non importé | partie 9.1 |
| `restaurer.sh` : « Refus : la cible contient déjà des comptes » | la cible n'est pas un projet neuf | protection voulue : utiliser un projet neuf |
| `restaurer.sh` : erreur sur `session_replication_role` | droit refusé sur ce projet | contacter le support Supabase ; ne pas forcer |
| `pg_dump: server version mismatch` | client PostgreSQL trop ancien | partie 1.3, installer une version ≥ serveur |

Si le problème persiste : noter la commande, le message complet et la partie du document, puis consulter `README_SUPABASE.md` (référence détaillée) ou le support Supabase.

---

## Annexe A — Où ranger chaque secret

| Secret | Où il doit être | Où il ne doit JAMAIS être |
|---|---|---|
| Clé `service_role` | tableau de bord Supabase ; `.env.test` / `.env.prod` locaux | site ( `js/` ), Git, e-mail, messagerie, captures d'écran |
| Mot de passe de la base / `DATABASE_URL` | gestionnaire de mots de passe ; `.env` local | Git, site |
| Mot de passe SMTP | tableau de bord Supabase (SMTP Settings) | Git, site, `.env` partagé |
| Secret Google OAuth | tableau de bord Supabase (Providers → Google) | Git, site |
| Clé de chiffrement des sauvegardes | gestionnaire de mots de passe | avec les sauvegardes elles-mêmes |

**Publique, sans danger** : l'URL du projet et la clé **anon / publishable** (elles sont dans le site par conception).

**Si une clé `service_role` a fuité** : Project Settings → API → régénérer les clés (*Roll* / *Generate new*), mettre à jour `.env`, redéployer `gerer-acces` (partie 7), mettre la nouvelle clé **anon** dans `js/00-config.js` et republier le site. Toutes les sessions sont alors fermées.

## Annexe B — Contenu du dossier

```
README_CONFIGURATION.md     ← CE DOCUMENT (suffit pour toute la configuration)
.env.example                modèle du fichier .env (partie 3)
js/00-config.js             le seul fichier du site à remplir (partie 8)
index.html, sw.js, manifest.webmanifest, css/, js/, img/   le site (partie 8.3)

supabase/
  migrations/               8 scripts SQL de la base (partie 5)
  functions/gerer-acces/    fonction serveur (partie 7)
  functions/assistant-ia/   assistant (relais Gemini, partie 7.2)
  templates/                modèles d'e-mails (partie 4.5)
  config.toml               réglages de référence (base locale)
  outils/importer-json.js   import du jeu de TEST (partie 9.1)
  outils/generer-schema-front.js   mise à jour du schéma du site (partie 13)
  sauvegarde/               sauvegarder.sh, restaurer.sh (parties 10, 11.4, 13)
  tests-local/              (dans DEUKSIL_OUTILS_DE_TEST.zip) test de sécurité, jeu de test
(partie 9)

tests/                      fiche de recette manuelle ; tests navigateur dans
DEUKSIL_OUTILS_DE_TEST.zip (partie 9)
docs/                       rapports détaillés (sécurité, performances, index) — lecture
facultative
README_SUPABASE.md          référence technique complète — lecture facultative
```

---

*DEUKSIL — README CONFIGURATION · 22 octobre 2026. Tous les scripts cités ont été exécutés avec succès sur une réplique locale de Supabase ; la configuration sur Supabase hébergé (parties 2 à 11) est celle que ce document vous fait réaliser et vérifier.*